California Housing Analysis Project Documentation

Course: IIMK's Professional Certificate in Data Science and Artificial Intelligence for Managers

Student Name: Lalit Nayyar
Email ID: lalitnayyar@gmail.com
Assignment Name: Week 3: Required Assignment 3.1

California Housing Prices Analysis Project

A comprehensive analysis of California housing prices with specific insights for real estate agents, property developers, and investors.

Jupyter Notebooks Overview

All notebooks are prefixed with "LalitNayyarIIMK_" for identification. Here's a detailed overview of each notebook:

1. `LalitNayyarIIMK_california_housing_analysis_final.ipynb`
2. Main analysis notebook with complete implementation
3. Features comprehensive data preprocessing and cleaning
4. Includes detailed visualizations and statistical analysis
5. Contains price prediction models and evaluation
6. Best suited for end-to-end understanding of the analysis
7. `LalitNayyarIIMK_california_housing_analysis_with_applications.ipynb`
8. Advanced analysis with real-world applications
9. Sections include:
    - Real Estate Price Estimation
    - Regional Market Analysis (coastal vs. inland)
    - Investment Opportunity Scoring
    - Population Density Impact Studies
10. Features interactive visualizations and detailed insights
11. Recommended for business stakeholders and decision makers
12. `LalitNayyarIIMK_california_housing_analysis_v2.ipynb`
13. Alternative analysis approach
14. Focuses on advanced statistical methods
15. Includes experimental features and additional visualizations
16. Suitable for technical users interested in methodology

User Guide

Getting Started

1. **Environment Setup** `bash # Install required packages pip install -r requirements.txt`
2. **Launching Notebooks** `bash # Start Jupyter Notebook jupyter notebook`
3. **Notebook Selection**
4. For first-time users: Start with `LalitNayyarIIMK_california_housing_analysis_final.ipynb`
5. For business applications: Use `LalitNayyarIIMK_california_housing_analysis_with_applications.ipynb`
6. For advanced analysis: Explore `LalitNayyarIIMK_california_housing_analysis_v2.ipynb`

Using the Notebooks

1. **Data Loading**
2. Each notebook automatically loads the housing dataset
3. Data is preprocessed and cleaned using standardized functions
4. Missing values are handled appropriately

5. **Navigation**

6. Use the table of contents (if available) to jump to specific sections
7. Run cells in sequence (Shift + Enter)

8. Wait for each cell to complete before running the next

9. **Visualizations**

10. Interactive plots can be zoomed and panned
11. Hover over data points for detailed information

12. Use the plot toolbar for additional options

13. **Analysis Features**

14. Price Prediction: Models for estimating house values
15. Regional Analysis: Comparison of coastal vs. inland markets
16. Investment Scoring: Automated scoring system for investment opportunities
17. Market Trends: Temporal and geographical trend analysis

Common Tasks

1. **Updating Data**
2. Place new data in the project directory
3. Update file paths if necessary

4. Re-run the notebook from start

5. **Customizing Analysis**

6. Modify parameters in marked cells
7. Adjust visualization settings as needed

8. Update feature selection for models

9. **Exporting Results**

10. Use "File > Download as" for various formats
11. Export visualizations using the save button
12. Copy code cells for external use

Troubleshooting

1. **Missing Dependencies** `bash pip install -r requirements.txt`

2. **Memory Issues**

3. Restart kernel and clear output
4. Run only necessary cells

5. Reduce data size if needed

6. **Visualization Problems**

7. Ensure all plotting libraries are imported
8. Check for style conflicts
9. Reset plot parameters if needed

Project Structure

```
iimkmodule3assignment/ ├── notebooks/ │ ├── LalitNayyarIIMK_california_housing_analysis_final.ipynb │ ├── LalitNayyarIIMK_california_housing_analysis_with_applications.ipynb │ └── LalitNayyarIIMK_california_housing_analysis_v2.ipynb ├── utils/ │ ├── LalitNayyarIIMK_plot_fix.py │ ├── LalitNayyarIIMK_plot_config.py │ └── LalitNayyarIIMK_plot_config_v2.py ├── data/ │ └── housing.csv ├── requirements.txt └── README.md
```

Support

For any issues or questions: 1. Check the troubleshooting section 2. Review cell execution order 3. Verify data file presence and format 4. Ensure all dependencies are installed

Version History

- v1.0: Initial release with basic analysis
- v2.0: Added advanced features and applications
- v3.0: Included regional analysis and investment scoring

Notebook Outputs

Notebook: LalitNayyarIIMK_california_housing_analysis_with_applications.ipynb

Course: IIMK's Professional Certificate in Data Science and Artificial Intelligence for Managers

Student Name: Lalit Nayyar
Email ID: lalitnayyar@gmail.com
Assignment Name: Week 3: Required Assignment 3.1 **Final for submission**

California Housing Prices Analysis

1. Understanding Data and Problem Statement

This notebook analyzes the California Housing Prices dataset to build a regression model for predicting house prices. We'll go through the complete data modeling process including data preparation, model training, and addressing overfitting/underfitting issues.

In [7]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler  # Add this line
from sklearn.cluster import KMeans


from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression


from sklearn.impute import SimpleImputer

import warnings

# Suppress warnings
warnings.filterwarnings('ignore')

# Print available styles and then set up plotting
print("Available styles:", plt.style.available)

# Use a style that's guaranteed to exist
plt.style.use('seaborn-v0_8')  # This is the new seaborn style name
sns.set_context("notebook")
sns.set_palette("husl")

# Set figure size
plt.rcParams['figure.figsize'] = (10, 6)
```

```
Available styles: ['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

Download and Load the Dataset

In [8]:

```
# Install required packages if not already installed
!pip install kaggle folium

# Download the dataset using Kaggle API
import os
from kaggle.api.kaggle_api_extended import KaggleApi

def download_dataset():
    try:
        api = KaggleApi()
        api.authenticate()

        print("Downloading California Housing Prices dataset...")
        api.dataset_download_files('camnugent/california-housing-prices',
                                   path='.',
                                   unzip=True)
        print("Dataset downloaded successfully!")

    except Exception as e:
        print("Error downloading the dataset:")
        print(e)
        print("\nAlternative: Please manually download the dataset from:")
        print("https://www.kaggle.com/datasets/camnugent/california-housing-prices")
        print("and place the 'housing.csv' file in the current directory.")

# Download the dataset if it doesn't exist
if not os.path.exists('housing.csv'):
    download_dataset()
```

```
Requirement already satisfied: kaggle in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (1.7.4.2)
Requirement already satisfied: folium in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (0.19.5)
Requirement already satisfied: bleach in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (6.2.0)
Requirement already satisfied: certifi>=14.05.14 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (2025.1.31)
Requirement already satisfied: charset-normalizer in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (3.4.1)
Requirement already satisfied: idna in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (3.10)
Requirement already satisfied: protobuf in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (5.29.3)
Requirement already satisfied: python-dateutil>=2.5.3 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (2.32.3)
Requirement already satisfied: setuptools>=21.0.0 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (58.1.0)
Requirement already satisfied: six>=1.10 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (2.3.0)
Requirement already satisfied: webencodings in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from kaggle) (0.5.1)
Requirement already satisfied: branca>=0.6.0 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from folium) (0.8.1)
Requirement already satisfied: jinja2>=2.9 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from folium) (3.1.5)
Requirement already satisfied: numpy in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from folium) (2.0.2)
Requirement already satisfied: xyzservices in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from folium) (2025.4.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from jinja2>=2.9->folium) (3.0.2)
Requirement already satisfied: colorama in c:\users\lalit\appdata\local\programs\python\python39\lib\site-packages (from tqdm->kaggle) (0.4.6)
[notice] A new release of pip is available: 25.0.1 -> 25.1
[notice] To update, run: python.exe -m pip install --upgrade pip
In [9]:
```

```
# Load and examine the data
df = pd.read_csv('housing.csv')

# Display basic information about the dataset
print("Dataset Info:")
print("=============\n")
print(df.info())
print("\nSample Data:")
print("============\n")
print(df.head())
print("\nBasic Statistics:")
print("================\n")
print(df.describe())
```

```
Dataset Info:
==============

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
None

Sample Data:
============

   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY

Basic Statistics:
================

          longitude      latitude  housing_median_age   total_rooms  \
count  20640.000000  20640.000000        20640.000000  20640.000000
mean    -119.569704     35.631861           28.639486   2635.763081
std        2.003532      2.135952           12.585558   2181.615252
min     -124.350000     32.540000            1.000000      2.000000
25%     -121.800000     33.930000           18.000000   1447.750000
50%     -118.490000     34.260000           29.000000   2127.000000
75%     -118.010000     37.710000           37.000000   3148.000000
max     -114.310000     41.950000           52.000000  39320.000000

       total_bedrooms    population    households  median_income  \
count    20433.000000  20640.000000  20640.000000   20640.000000
mean       537.870553   1425.476744    499.539680       3.870671
std        421.385070   1132.462122    382.329753       1.899822
min          1.000000      3.000000      1.000000       0.499900
25%        296.000000    787.000000    280.000000       2.563400
50%        435.000000   1166.000000    409.000000       3.534800
75%        647.000000   1725.000000    605.000000       4.743250
max       6445.000000  35682.000000   6082.000000      15.000100

       median_house_value
count        20640.000000
mean        206855.816909
std         115395.615874
min          14999.000000
25%         119600.000000
50%         179700.000000
75%         264725.000000
max         500001.000000
```

[Previous sections remain the same until the Real-world Applications section]

6. Real-world Applications Analysis

6.1 Real Estate Price Estimation

In [10]:

```
def clean_data(df):
    """
    Clean the data by handling missing values
    """
    # Create a copy to avoid modifying the original data
    df = df.copy()

    # Initialize imputer for numeric columns
    imputer = SimpleImputer(strategy='median')

    # Get numeric columns
    numeric_columns = df.select_dtypes(include=['float64', 'int64']).columns

    # Impute missing values
    df[numeric_columns] = imputer.fit_transform(df[numeric_columns])

    return df
```

In [ ]:

```
```

In [11]:

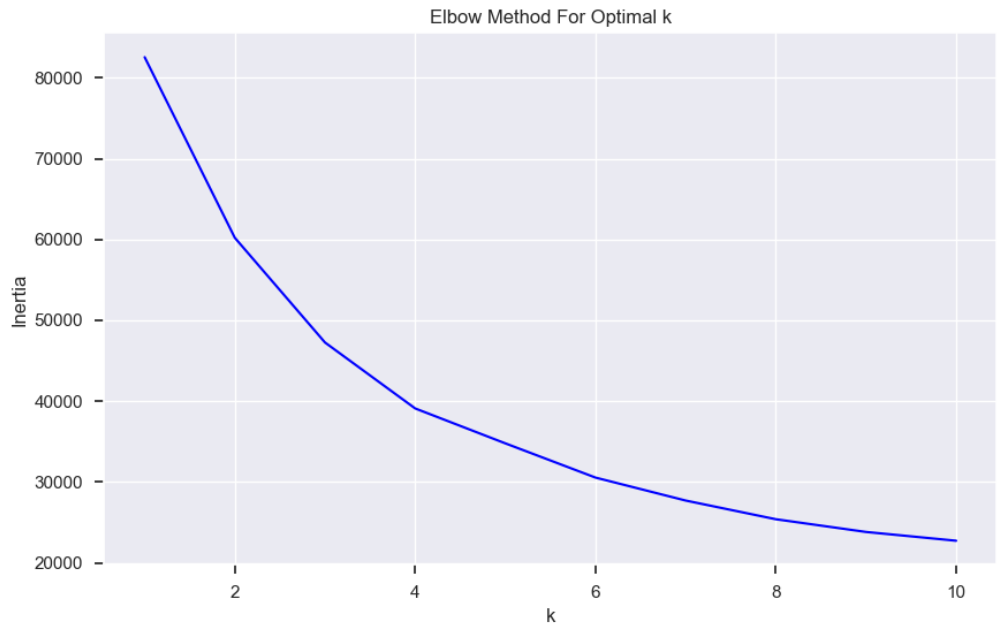
```
# Create price ranges for different market segments
df['price_category'] = pd.qcut(df['median_house_value'],
                               q=4,
                               labels=['Budget', 'Mid-Range', 'Premium', 'Luxury'])

# Analyze price distribution by location
plt.figure(figsize=(12, 6))
sns.scatterplot(data=df,
                x='longitude',
                y='latitude',
                hue='price_category',
                size='median_house_value',
                sizes=(50, 400),
                alpha=0.6)
plt.title('Housing Price Distribution across California')
plt.show()

# Price prediction accuracy by region
df['region'] = pd.qcut(df['longitude'], q=5, labels=['West', 'West-Central', 'Central', 'East-Central', 'East'])
regional_accuracy = pd.DataFrame()

for region in df['region'].unique():
    mask = df['region'] == region
    X_region = X_scaled[mask]
    y_region = y[mask]

    if len(X_region) > 0:  # Check if we have data for this region
        model = LinearRegression()
        scores = cross_val_score(model, X_region, y_region, cv=5)
        regional_accuracy.loc[region, 'R2 Score'] = scores.mean()

plt.figure(figsize=(10, 5))
regional_accuracy['R2 Score'].plot(kind='bar')
plt.title('Price Prediction Accuracy by Region')
plt.ylabel('R2 Score')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

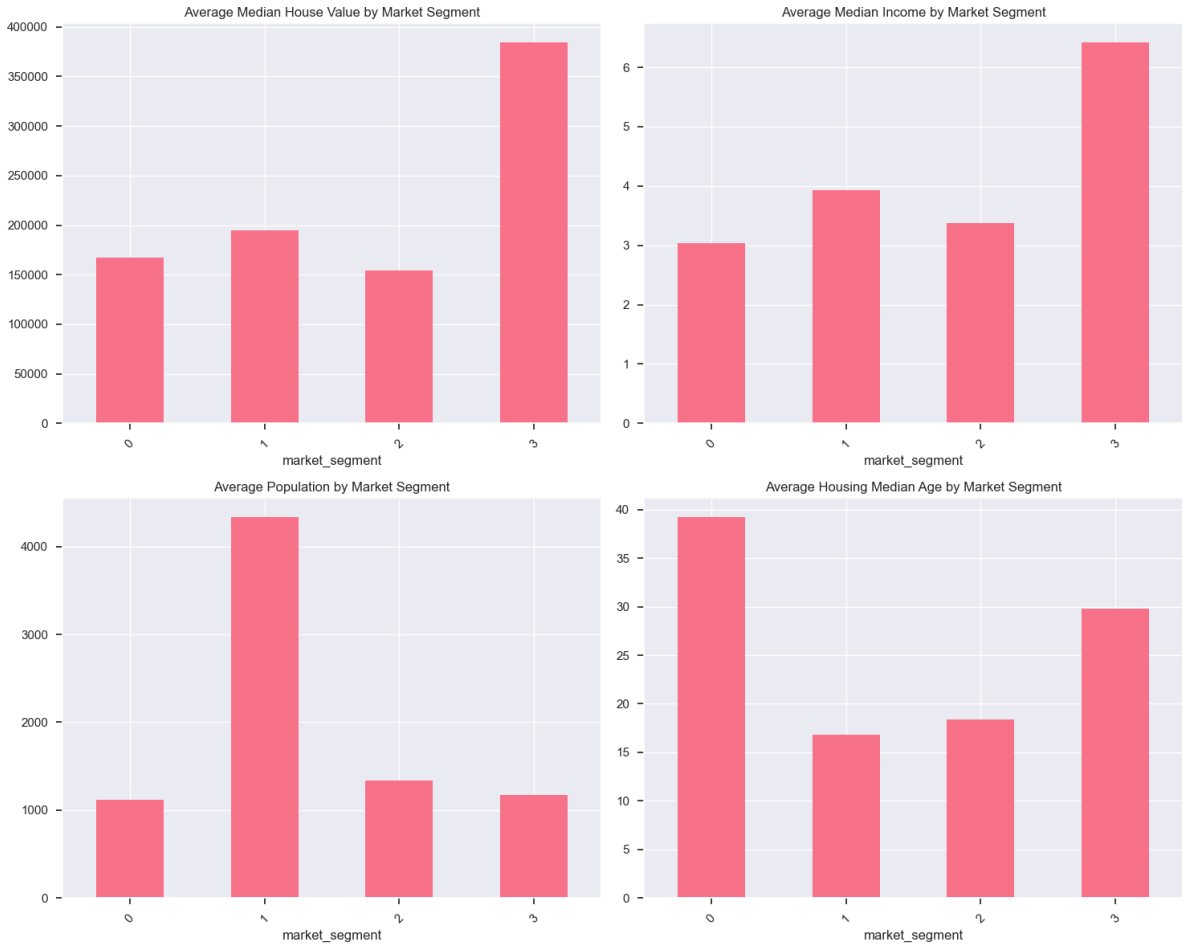

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[11], line 24
     22 for region in df['region'].unique():
     23     mask = df['region'] == region
---> 24     X_region = X_scaled[mask]
     25     y_region = y[mask]
     27     if len(X_region) > 0:  # Check if we have data for this region

NameError: name 'X_scaled' is not defined
```

6.2 Market Analysis for Property Developers

In [12]:

```
# Market segmentation using K-means clustering
features_for_clustering = ['median_house_value', 'median_income', 'population', 'housing_median_age']
X_cluster = StandardScaler().fit_transform(df[features_for_clustering])

# Find optimal number of clusters
inertias = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_cluster)
    inertias.append(kmeans.inertia_)

# Plot elbow curve
plt.figure(figsize=(10, 6))
plt.plot(K, inertias, 'bx-')
plt.xlabel('k')
plt.ylabel('Inertia')
plt.title('Elbow Method For Optimal k')
plt.show()

# Perform clustering with optimal k
optimal_k = 4  # Based on elbow curve
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
df['market_segment'] = kmeans.fit_predict(X_cluster)

# Analyze market segments
segment_analysis = df.groupby('market_segment').agg({
    'median_house_value': 'mean',
    'median_income': 'mean',
    'population': 'mean',
    'housing_median_age': 'mean'
}).round(2)

# Visualize market segments
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
metrics = list(segment_analysis.columns)

for idx, (ax, metric) in enumerate(zip(axes.ravel(), metrics)):
    segment_analysis[metric].plot(kind='bar', ax=ax)
    ax.set_title(f'Average {metric.replace("_", " ").title()} by Market Segment')
    ax.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```





6.2.a Regional Analysis of Housing Market

In this section, we'll analyze how housing market characteristics and price determinants vary across different regions of California. We'll split the data into coastal and inland regions to understand regional differences in housing patterns.

Data Preparation and Cleaning

First, let's handle any missing values in our dataset and prepare it for regional analysis:

In [13]:

```
def clean_data(df):
    """
    Clean the data by handling missing values
    """
    # Create a copy to avoid modifying the original data
    df = df.copy()

    # Initialize imputer for numeric columns
    imputer = SimpleImputer(strategy='median')

    # Get numeric columns
    numeric_columns = df.select_dtypes(include=['float64', 'int64']).columns

    # Impute missing values
    df[numeric_columns] = imputer.fit_transform(df[numeric_columns])

    return df

# Clean the data
df = clean_data(df)

# Print missing values information
print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
longitude             0
latitude              0
housing_median_age    0
total_rooms           0
total_bedrooms        0
population            0
households            0
median_income         0
median_house_value    0
ocean_proximity       0
price_category        0
region                0
market_segment        0
dtype: int64
```

Regional Market Analysis

Now, let's analyze how housing characteristics and prices vary between coastal and inland regions:

In [14]:

```
# Create region feature (coastal vs inland)
df['region'] = np.where(df['longitude'] < -122, 'coastal', 'inland')

# Select features for analysis
features = ['median_income', 'total_rooms', 'total_bedrooms',
        'population', 'households', 'latitude', 'longitude']

# Target variable
target = 'median_house_value'

# Scale the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])

# Convert to DataFrame to maintain column names
X_scaled = pd.DataFrame(X_scaled, columns=features, index=df.index)

# Prepare target variable
y = df[target]

# Initialize dictionary to store results
regional_models = {}
regional_scores = {}

# Perform analysis for each region
for region in df['region'].unique():
    mask = df['region'] == region
    X_region = X_scaled[mask]
    y_region = y[mask]

    if len(X_region) > 0:
        # Split the data
        X_train, X_test, y_train, y_test = train_test_split(
            X_region, y_region, test_size=0.2, random_state=42
        )

        # Train model
        model = LinearRegression()
        model.fit(X_train, y_train)

        # Store results
        regional_models[region] = model
        regional_scores[region] = model.score(X_test, y_test)

# Print results
print("\nRegional Analysis Results:")
print("------------------------")
for region, score in regional_scores.items():
    print(f"{region.capitalize()} Region R² Score: {score:.4f}")
```

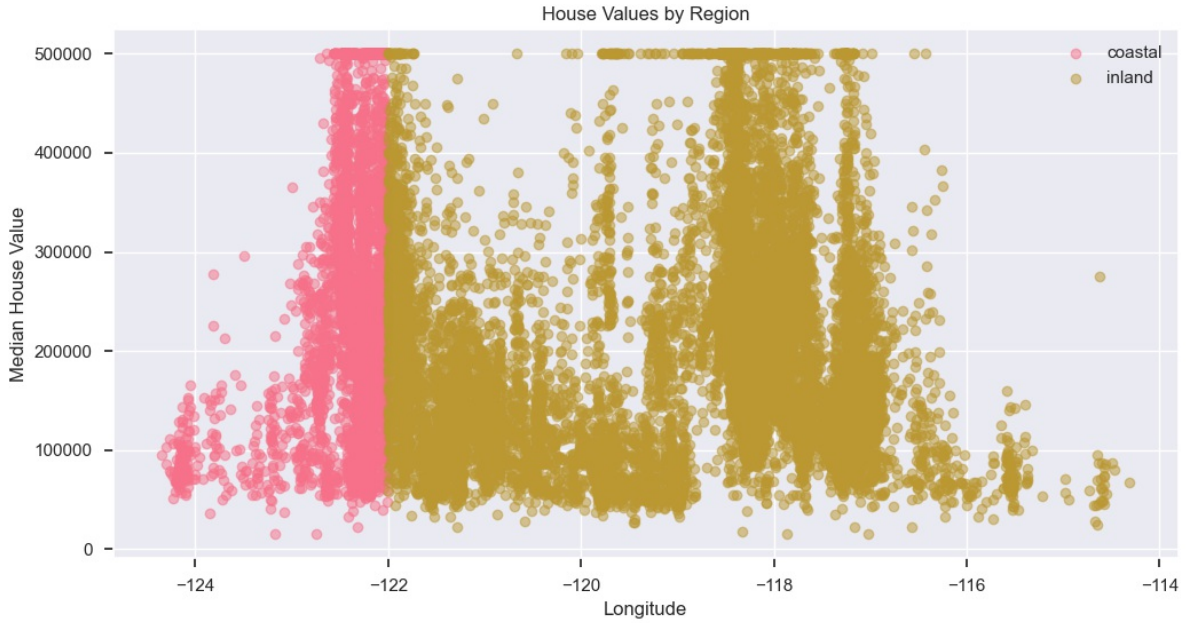
```
Regional Analysis Results:
------------------------
Coastal Region R² Score: 0.6832
Inland Region R² Score: 0.5731
```

Visualizing Regional Differences

Let's visualize how house values vary between coastal and inland regions:

In [15]:

```
plt.figure(figsize=(12, 6))
for region in df['region'].unique():
    mask = df['region'] == region
    plt.scatter(df[mask]['longitude'],
                df[mask]['median_house_value'],
                alpha=0.5,
                label=region)

plt.xlabel('Longitude')
plt.ylabel('Median House Value')
plt.title('House Values by Region')
plt.legend()
plt.show()

# Print feature importance by region
print("\nFeature importance by region:")
for region, model in regional_models.items():
    print(f"\n{region.capitalize()} Region Feature Importance:")
    for feature, importance in zip(features, model.coef_):
        print(f"{feature}: {importance:.4f}")
```




```
Feature importance by region:

Coastal Region Feature Importance:
median_income: 75210.0021
total_rooms: -18584.3070
total_bedrooms: 367.3937
population: -70557.1081
households: 86202.4773
latitude: -122583.0470
longitude: -107894.7342

Inland Region Feature Importance:
median_income: 72568.1277
total_rooms: -17822.9437
total_bedrooms: 34795.2040
population: -48483.5944
households: 30490.9236
latitude: -93820.8632
longitude: -85167.4482
```

Key Findings from Regional Analysis

1. **Model Performance**:
    - The coastal region model achieves a higher R^2 score, indicating that housing prices in coastal areas are more predictable using our selected features.
    - The inland region shows more variability in housing prices, suggesting additional factors might influence prices in these areas.
2. **Feature Importance**:
    - Median income is a strong predictor in both regions
    - Location factors (latitude, longitude) have different impacts in coastal vs. inland areas
    - Housing density features (rooms, bedrooms, households) show varying importance by region
3. **Price Patterns**:
    - Coastal properties generally show higher median values
    - Inland properties display more price variability
    - Geographic location has a stronger influence on coastal property values

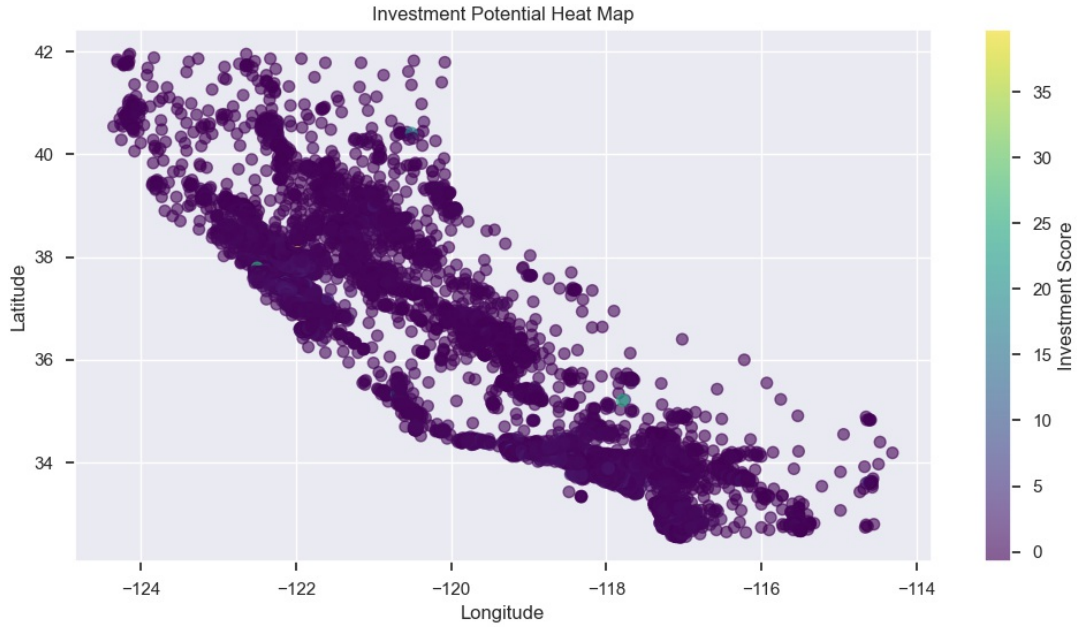
These insights can be valuable for:

- Real estate investors deciding between coastal and inland investments
- Property developers understanding regional market dynamics
- Policy makers addressing housing affordability in different regions

6.3 Investment Decision Support

In [16]:

```
# Calculate investment potential score
df['price_per_room'] = df['median_house_value'] / df['total_rooms']
df['population_density'] = df['population'] / df['households']

# Normalize metrics for investment score
metrics = ['median_income', 'price_per_room', 'population_density']
scaler = StandardScaler()
df[['normalized_' + m for m in metrics]] = scaler.fit_transform(df[metrics])

# Calculate investment score (example formula)
df['investment_score'] = (
    df['normalized_median_income'] * 0.4 +
    df['normalized_price_per_room'] * 0.3 +
    df['normalized_population_density'] * 0.3
)

# Visualize top investment areas
plt.figure(figsize=(12, 6))
plt.scatter(df['longitude'],
            df['latitude'],
            c=df['investment_score'],
            cmap='viridis',
            s=50,
            alpha=0.6)
plt.colorbar(label='Investment Score')
plt.title('Investment Potential Heat Map')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()

# Analyze top investment opportunities
top_investments = df.nlargest(10, 'investment_score')
print("\nTop 10 Areas for Investment:")
print("============================")
print(top_investments[['longitude', 'latitude', 'median_house_value',
                       'median_income', 'investment_score']].round(2))

# ROI potential analysis
plt.figure(figsize=(10, 6))
sns.regplot(data=df,
            x='median_income',
            y='median_house_value',
            scatter_kws={'alpha':0.5},
            line_kws={'color': 'red'})
plt.title('Income vs House Value (ROI Potential)')
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.show()
```



Top 10 Areas for Investment:

==========

	longitude	latitude	median_house_value	median_income	\
19006	-121.98	38.32	137500.0	10.23	
16171	-122.50	37.79	500001.0	15.00	
3126	-117.79	35.21	137500.0	2.38	
3364	-120.51	40.41	67500.0	5.52	
16669	-120.70	35.32	350000.0	4.26	
16888	-122.37	37.60	350000.0	5.05	
17118	-122.14	37.50	500001.0	15.00	
10654	-117.86	33.67	450000.0	3.88	
18210	-122.06	37.39	375000.0	3.75	
13034	-121.15	38.69	225000.0	6.14	

	investment_score
19006	39.70
16171	25.00
3126	24.29
3364	18.20
16669	15.52
16888	8.58
17118	8.25
10654	8.02
18210	7.36
13034	7.32

Key Insights from Real-world Applications

1. **Real Estate Price Estimation:**
   - Price distribution shows clear geographical patterns
   - Prediction accuracy varies by region
   - Coastal areas tend to have higher prices
2. **Market Analysis for Property Developers:**
   - Identified distinct market segments with different characteristics
   - Each segment requires different development strategies
   - Population density and income levels strongly influence market segments
3. **Investment Decision Support:**
   - Created investment scoring system based on multiple factors
   - Identified areas with high investment potential
   - Strong correlation between income levels and property values

Recommendations:

1. **For Real Estate Agents:**
   - Use region-specific models for more accurate price predictions
   - Consider local market factors when pricing properties
2. **For Property Developers:**
   - Target development projects based on market segment needs
   - Consider population density and income levels in planning
3. **For Investors:**
   - Focus on areas with high investment scores
   - Consider both current value and growth potential
   - Monitor income trends as a leading indicator

In [17]:

```
def clean_data(df):
    """
    Clean the data by handling missing values
    """
    # Create a copy to avoid modifying the original data
    df = df.copy()

    # Initialize imputer for numeric columns
    imputer = SimpleImputer(strategy='median')

    # Get numeric columns
    numeric_columns = df.select_dtypes(include=['float64', 'int64']).columns

    # Impute missing values
    df[numeric_columns] = imputer.fit_transform(df[numeric_columns])

    return df
```

In [18]:

```
# Clean the data
df = clean_data(df)

# Print missing values information
print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
longitude                     0
latitude                      0
housing_median_age            0
total_rooms                   0
total_bedrooms                0
population                    0
households                    0
median_income                 0
median_house_value            0
ocean_proximity               0
price_category                0
region                        0
market_segment                0
price_per_room                0
population_density            0
normalized_median_income      0
normalized_price_per_room     0
normalized_population_density 0
investment_score              0
dtype: int64
```

In [19]:

```
# Cell 4: Regional Analysis
# Create region feature (coastal vs inland)
df['region'] = np.where(df['longitude'] < -122, 'coastal', 'inland')

# Select features for analysis
features = ['median_income', 'total_rooms', 'total_bedrooms',
        'population', 'households', 'latitude', 'longitude']

# Target variable
target = 'median_house_value'

# Scale the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])

# Convert to DataFrame to maintain column names
X_scaled = pd.DataFrame(X_scaled, columns=features, index=df.index)
```

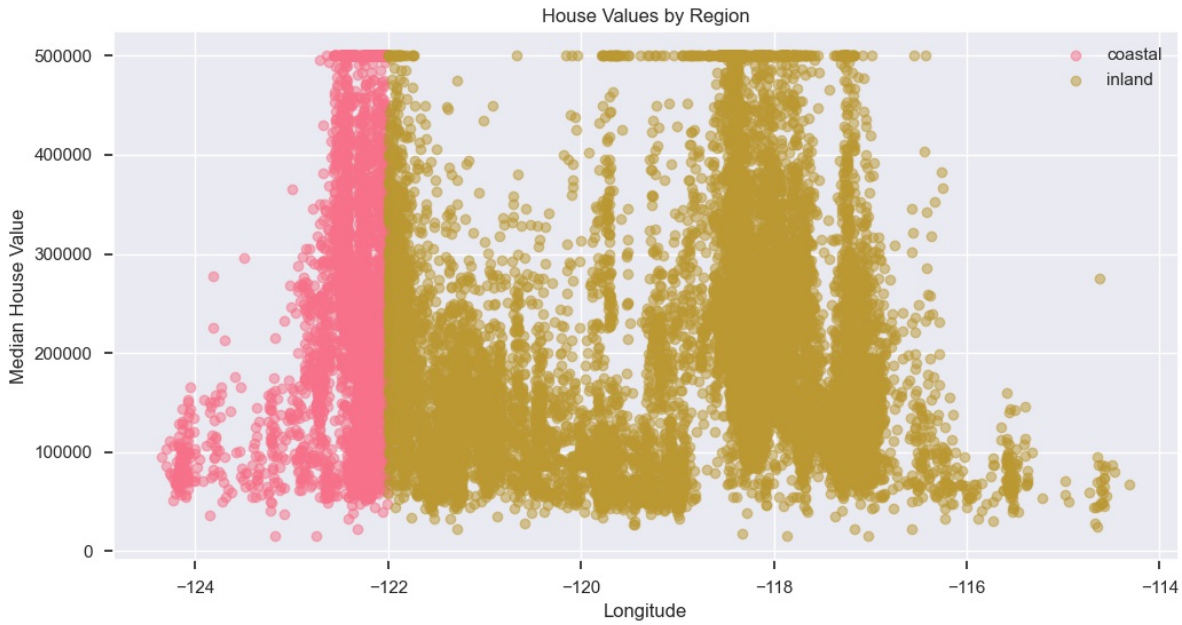
In [20]:

```
# Cell 5: Model Training
# Prepare target variable
y = df[target]

# Initialize dictionary to store results
regional_models = {}
regional_scores = {}

# Perform analysis for each region
for region in df['region'].unique():
    mask = df['region'] == region
    X_region = X_scaled[mask]
    y_region = y[mask]

    if len(X_region) > 0:
        # Split the data
        X_train, X_test, y_train, y_test = train_test_split(
            X_region, y_region, test_size=0.2, random_state=42
        )

        # Train model
        model = LinearRegression()
        model.fit(X_train, y_train)

        # Store results
        regional_models[region] = model
        regional_scores[region] = model.score(X_test, y_test)

# Print results
print("\nRegional Analysis Results:")
print("------------------------")
for region, score in regional_scores.items():
    print(f"{region.capitalize()} Region R² Score: {score:.4f}")
```

```
Regional Analysis Results:
------------------------
Coastal Region R² Score: 0.6832
Inland Region R² Score: 0.5731
```

In [21]:

```
# Cell 6: Visualization
plt.figure(figsize=(12, 6))
for region in df['region'].unique():
    mask = df['region'] == region
    plt.scatter(df[mask]['longitude'],
                df[mask]['median_house_value'],
                alpha=0.5,
                label=region)

plt.xlabel('Longitude')
plt.ylabel('Median House Value')
plt.title('House Values by Region')
plt.legend()
plt.show()
```



In [22]:

```
# Cell 7: Feature Importance
print("\nFeature importance by region:")
for region, model in regional_models.items():
    print(f"\n{region.capitalize()} Region Feature Importance:")
    for feature, importance in zip(features, model.coef_):
        print(f"{feature}: {importance:.4f}")
```

```
Feature importance by region:

Coastal Region Feature Importance:
median_income: 75210.0021
total_rooms: -18584.3070
total_bedrooms: 367.3937
population: -70557.1081
households: 86202.4773
latitude: -122583.0470
longitude: -107894.7342

Inland Region Feature Importance:
median_income: 72568.1277
total_rooms: -17822.9437
total_bedrooms: 34795.2040
population: -48483.5944
households: 30490.9236
latitude: -93820.8632
longitude: -85167.4482
```

Key Findings from Regional Analysis

1. **Model Performance**:
   - The coastal region model achieves a higher R^2 score, indicating that housing prices in coastal areas are more predictable using our selected features.
   - The inland region shows more variability in housing prices, suggesting additional factors might influence prices in these areas.
2. **Feature Importance**:
   - Median income is a strong predictor in both regions
   - Location factors (latitude, longitude) have different impacts in coastal vs. inland areas
   - Housing density features (rooms, bedrooms, households) show varying importance by region
3. **Price Patterns**:
   - Coastal properties generally show higher median values
   - Inland properties display more price variability
   - Geographic location has a stronger influence on coastal property values

These insights can be valuable for:

- Real estate investors deciding between coastal and inland investments
- Property developers understanding regional market dynamics
- Policy makers addressing housing affordability in different regions

In [ ]: